

# Algorithmen und Datenstrukturen | AlgDat

## Zusammenfassung

---

## INHALTSVERZEICHNIS

|                                      |          |
|--------------------------------------|----------|
| <b>1. Suchbäume</b>                  | <b>2</b> |
| <b>1.1. Binärer Suchbaum</b>         | <b>2</b> |
| 1.1.1. Traversierung                 | 2        |
| 1.1.2. Suche                         | 2        |
| 1.1.3. Einfügen                      | 2        |
| 1.1.4. Löschen                       | 3        |
| 1.1.5. Performance                   | 3        |
| 1.1.6. Arithmetische Progression     | 3        |
| <b>1.2. AVL Baum</b>                 | <b>3</b> |
| 1.2.1. Einfügen                      | 3        |
| 1.2.2. Löschen                       | 5        |
| 1.2.3. Performance                   | 5        |
| <b>1.3. Splay Baum</b>               | <b>5</b> |
| 1.3.1. Löschen                       | 6        |
| 1.3.2. Splay Bäume und (Multi-) Maps | 6        |
| 1.3.3. Performance                   | 6        |
| <b>2. Sorting</b>                    | <b>6</b> |
| <b>3. Text Processing</b>            | <b>6</b> |
| <b>4. Graphs</b>                     | <b>6</b> |

# 1. SUCHBÄUME

**Multimaps:** Ungeordnet, `find(k)`, `findAll(k)`, `insert(k,o)`, `remove(e)`

**Geordnete Multimaps:** Geordnet, `first()`, `last()`, `successors(k)`, `predecessors(k)`

**Binäre Suche:** Bei einer Multimap, realisiert als array-basierte Sequenz, sortiert nach Key, bei `find(k)`: bei jedem Schritt wird die Anzahl der Kandidaten halbiert, terminiert nach  $O(\log n)$  Schritten

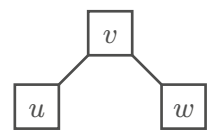
**Suchtabelle:** Multimap, welche mithilfe einer sortierten Sequenz implementiert wird.

- `find`:  $O(\log n)$
- `insert`:  $O(n)$
- `remove`:  $O(n)$

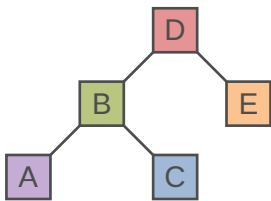
## 1.1. BINÄRER SUCHBAUM

Ein binärer Baum, welcher Keys (oder Key- Value-Entries) in seinen internen Knoten speichert und folgende Bedingungen erfüllt:

Gegeben sind die drei Knoten  $u$ ,  $v$ , und  $w$ .  $u$  ist im linken Teilbaum von  $v$  und  $w$  ist im rechten Teilbaum von  $v$ . So gilt:  $\text{key}(u) \leq \text{key}(v) \leq \text{key}(w)$



### 1.1.1. Traversierung



Preorder (W-L-R)  
 Postorder (L-R-W)  
 Inorder (L-W-R)  
 Breadth-First/ Level-Order

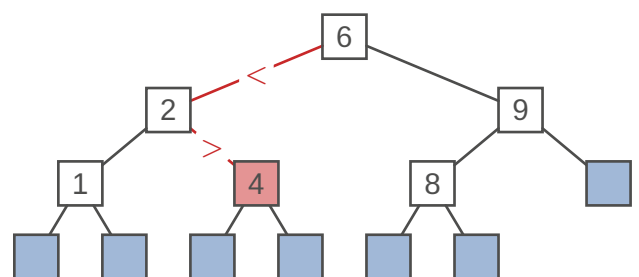
|   |   |   |   |   |
|---|---|---|---|---|
| D | B | A | C | E |
| A | C | B | E | D |
| A | B | C | D | E |
| D | B | E | A | C |

### 1.1.2. Suche

```

V search(K k, Node<V> n) {
  if (n.isExternal)
    return null;
  else if (k < n.key)
    return search(k, n.left);
  else if (k > n.key)
    return search(k, n.right);
  else
    return n.value;
}
  
```

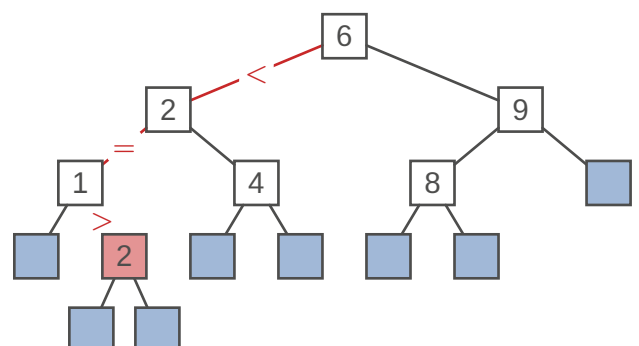
`search(4)`



### 1.1.3. Einfügen

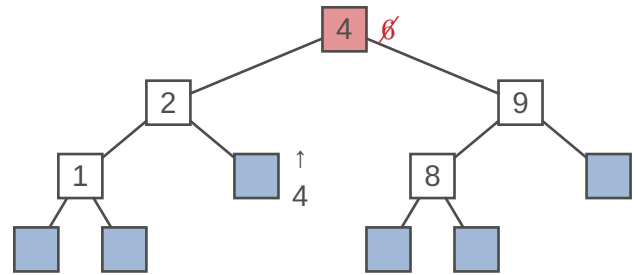
Annahme: der Baum ist eine Multimap und  $k$  ist schon im Baum vorhanden: Im jeweils linken Teilbaum von  $k$  wird weitergesucht bis man auf ein Blattknoten  $w$  stösst

`insert(2)`



#### 1.1.4. Löschen

- finde den internen Knoten  $w$  (4), welchem  $v$  (6) in der `delete(6)` Inorder-Traversierung folgt
- kopiere  $\text{key}(w)$  in den Knoten  $v$
- lösche den Knoten  $w$  und sein rechtes Kind  $z$  (welches ein Blatt sein muss)



#### 1.1.5. Performance

Betrachte eine Map mit  $n$  Entries, welcher mit einem binären Suchbaum implementiert ist mit der Höhe  $h$

- nötiger Speicher ist  $O(n)$
- Methoden `find`, `insert` und `remove` benötigen  $O(h)$
- Die Höhe  $h$  ist  $O(n)$  im schlechtesten Fall und  $O(\log n)$  im besten Fall

#### 1.1.6. Arithmetische Progression

Der Worst-Case tritt z.B. ein, wenn man aufsteigend sortierte Werte in einen binären Such-Baum einfügt. Laut der **Gauss'schen Summenformel**  $\frac{n(n+1)}{2}$  ist die Worst-Case Laufzeit somit  $O(n^2)$ .

**TODO:**

implementation?

## 1.2. AVL BAUM

Ein AVL Baum ist ein binärer Such-Baum, bei dem für jeden internen Knoten  $v$  von  $T$  gilt: die Höhe der Kinder von  $v$  unterscheiden sich höchstens um 1. AVL Bäume sind balanciert.

Die Höhe eines AVL Baumes  $T$ , der  $n$  Keys speichert, ist  $O(\log(n))$ . Die **minimale** Anzahl Knoten können wir als Funktion der Höhe  $h$  definieren:

$$n(1) = 1$$

$$n(2) = 2$$

$$n(h) = 1 + n(h-1) + n(h-2)$$

Dabei gilt

$$n(h-1) > n(h-2)$$

$$n(h) > 2n(h-2)$$

$$n(h) > 2^{\frac{h}{2}-1}$$

$$h < 2 \log(n(h)) + 2$$

$$\Rightarrow \text{Höhe} \sim O(\log(n))$$

Balance

$$b(k) = \text{Höhe(links)} - \text{Höhe(rechts)}$$

$$b(k) \in \{-1, 0, 1\}$$

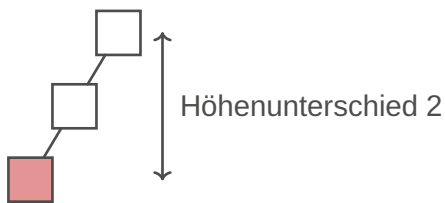
$$-2 \leq b(k) \leq 2$$

Nach dem Einfügen eines neuen Knotens

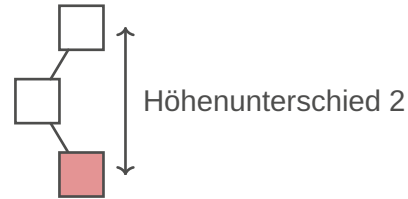
#### 1.2.1. Einfügen

Wie beim binären Such-Baum. Verletzungen des AVL-Merkmals können beispielsweise auftreten beim:

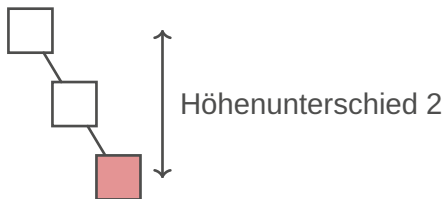
1) Einfügen eines Knotens in den linken Teilbaum des linken Sohnes



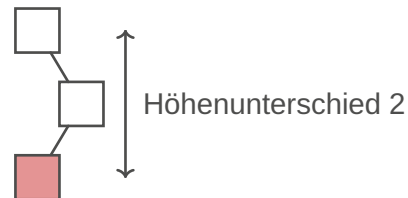
2) Einfügen eines Knotens in den rechten Teilbaum des linken Sohnes



3) Einfügen eines Knotens in den rechten Teilbaum des rechten Sohnes



4) Einfügen eines Knotens in den linken Teilbaum des rechten Sohnes



Nach dem Einfügen wandern wir vom neuen Knoten aus aufwärts, bis wir den ersten Knoten  $x$  finden, dessen Grosseltern  $z$  ein unbalancierter Knoten ist und balancieren den Baum aus.

#### 1.2.1.1. Rotieren

```

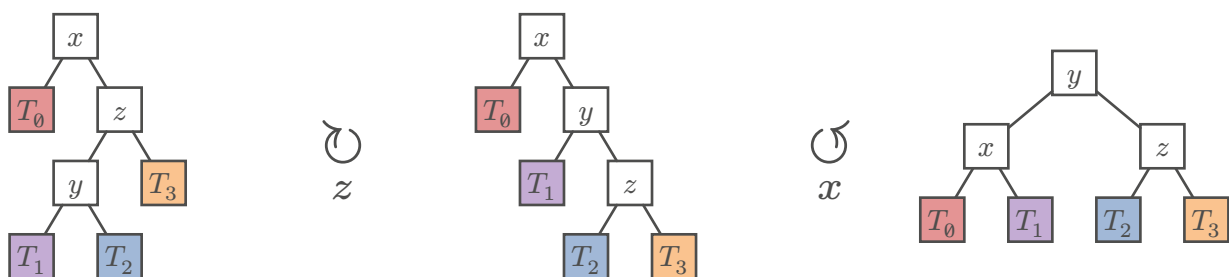
BinaryNode rotateWithLeftChild (BinaryNode k2) {
    BinaryNode k1 = k2.left;
    k2.left = k1.right;
    k1.right = k2;
    return k1;
}

```

Im Beispiel unten wird diese Rotation mit  $k2 = z$  und  $k1 = y$  durchgeführt.

#### 1.2.1.2. Cut/Link Restrukturierungs-Algorithmus

Sei  $(a, b, c)$  die (Inorder) geordnete Liste der Knoten  $x, y, z$ , und sei  $(T_0, T_1, T_2, T_3)$  die Liste der vier Unterbäume von  $x, y, z$ .

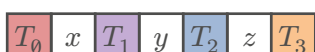


Ab Zustand 2:

- 1) Ersetze den Unterbaum mit Root  $x$  durch den Unterbaum mit Root  $y$ .
- 2) Setze  $x$  als linkes Kind von  $y$  und  $T_0, T_1$  als den linken resp. rechten Unterbaum von  $x$ .
- 3) Setze  $z$  als rechtes Kind von  $y$  und  $T_2, T_3$  als den linken resp. rechten Unterbaum von  $z$ .

Alternativ:

- 1) Kreiere ein Array mit 7 Elementen und befülle es In-Order mit den Knoten und den Unterbäumen:



- 2) Baue den Baum schrittweise wieder auf (beginnend bei  $y$ )

### 1.2.2. Löschen

Das Löschen eines Knotens  $w$  beginnt wie im binären Suchbaum. Sein Eltern-Knoten kann jetzt die Balance aus dem Gleichgewicht bringen, somit muss eine Umstrukturierung stattfinden. Die Umstrukturierung kann eine neue Unbalance hervorrufen bei Knoten höher im Baum. Somit muss die Balance weiter geprüft werden bis die Wurzel von  $T$  erreicht ist.

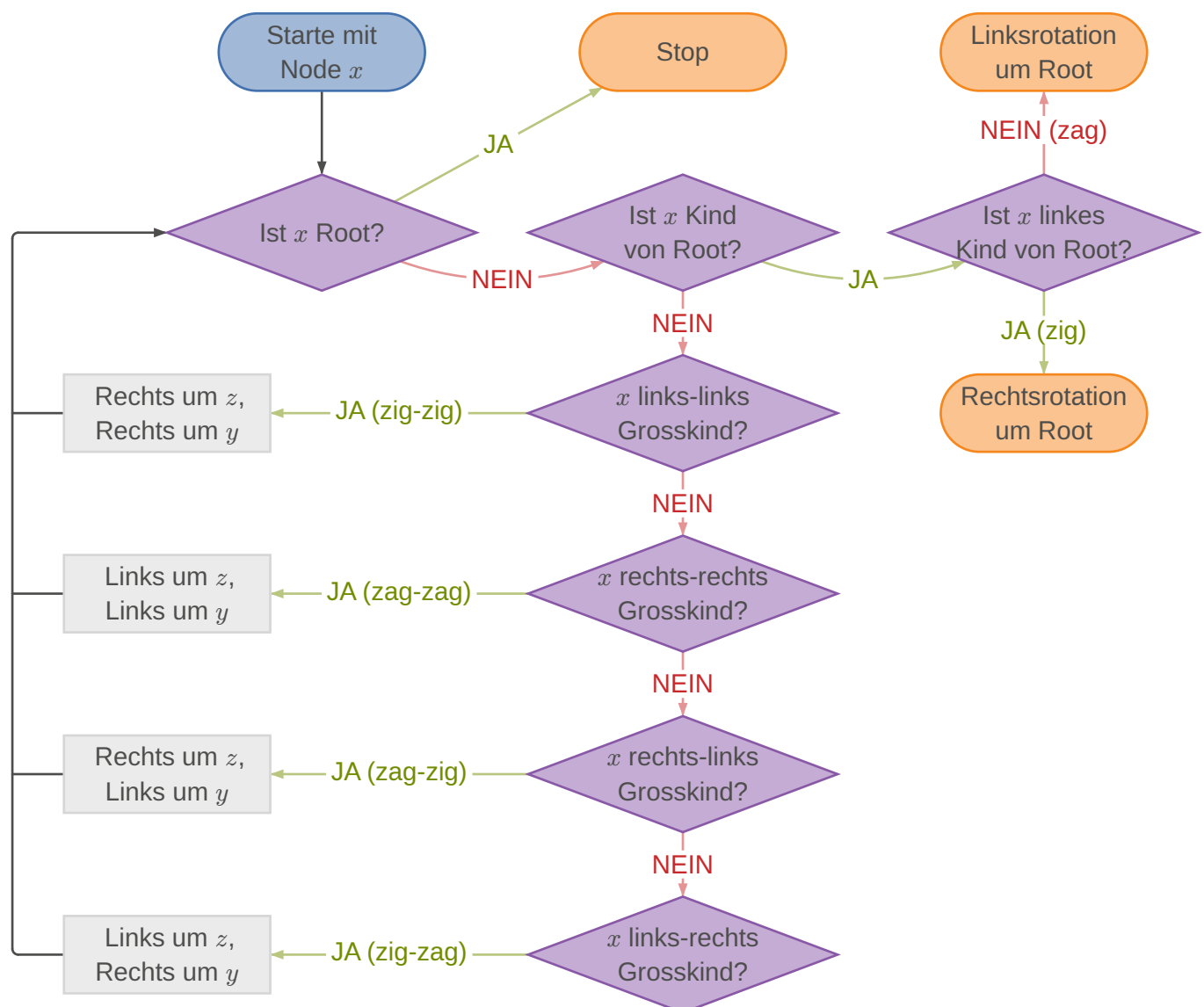
### 1.2.3. Performance

- Umstrukturierung ist  $O(1)$
- `find` ist  $O(\log n)$  (= Höhe des Baumes)
- `insert` ist  $O(\log n)$  (`find` zu Beginn ist  $O(\log n)$ , eventuelle Restrukturierungen  $O(1)$ )
- `delete` ist  $O(\log n)$  (`find` zu Beginn sowie eventuelle Restrukturierungen sind  $O(\log n)$ )

## 1.3. SPLAY BAUM

Ein Splay Baum ist ein binärer Such-Baum, bei welchem nach einem Zugriff auf einen Knoten dieser zur Root bewegt wird (jede Operation, auch Suche). Diese neue Operation heisst `splay`. Bewegt einen Knoten zum Root unter Benutzung von Rotationen.

« $x$  ist das links-rechts Grosskind»:  $x$  ist das linke Kind von seinem Eltern-Knoten, welcher selber ein rechtes Kind ist von seinem Eltern-Knoten.  $y$  ist  $x$ 's Eltern-Knoten;  $z$  ist  $y$ 's Eltern-Knoten



### 1.3.1. Löschen

**TODO:**

W3 S12 diagrams

### 1.3.2. Splay Bäume und (Multi-) Maps

Welcher Knoten wird «splayed» nach jeder Operation?

| Methode                   | Splay Knoten                                                                                                               |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <code>find(k)</code>      | Wenn Key gefunden, benutze diesen Knoten<br>Wenn Key nicht gefunden, benutze den Eltern-Knoten des externen Knoten am Ende |
| <code>insert(k, v)</code> | Benutze den neuen Knoten bei welchem der Entry eingefügt/ersetzt wurde                                                     |
| <code>remove(k)</code>    | Benutze den Eltern-Knoten des internen Knotens welcher gelöscht wurde                                                      |

### 1.3.3. Performance

- `splay` ist  $O(h)$ 
  - Durchschnittlich:  $O(\log n)$ , Für oft besuchte Knoten wesentlich schneller
  - Worst-Case:  $O(h)$  Rotationen, jede mit  $O(1)$

---

## 2. SORTING

---

## 3. TEXT PROCESSING

---

## 4. GRAPHS